

FIRST SEMESTER · B.TECH / BCA

Introduction to Programming using C

Complete Study Material — Theory + Practical, Basic to Advanced

50 Daily Classes (40 min each)

6 Modules

150+ Worked Problems

Beginner Friendly

Course: Introduction to Programming using C | **L-T-P:** 3-0-0 | **Credits:** 3

For: B.Tech (CSE / CSE-AI / CSE-DS / CSE-CC) & BCA — 1st Year, 1st Semester

Designed so a complete beginner can follow, one class at a time.

How to Use This Study Material

Welcome! This booklet turns your entire C programming syllabus into **50 short daily classes**. Each class is planned for about **40 minutes** and is marked **Day 1, Day 2**, and so on. The days are grouped into **6 Modules** that exactly match your official syllabus, plus a starter **Module 0** that teaches the very basics (what a program is, the history of C, and how to install the tools).

Each day is built in the same simple pattern:

- **Goal of the day** — one line telling you what you will be able to do by the end.
- **Theory in simple English** — the idea explained the way a friend would explain it.
- **Worked code + Output** — real C programs you can type and run yourself.
- **Practice Problems** — solved questions with a *How it works* explanation.

Tip: Do not just read the code — **type it and run it**. Programming is a skill like cycling; you learn it by doing, not by watching. If a program gives an error, that is normal. Read the error, fix it, and try again.

Symbols used: the blue **In simple words** boxes give an easy explanation, green boxes show the **Output** of a program, and red **Watch out** boxes warn you about common beginner mistakes.

Contents at a Glance

MODULE 0 Getting Started — Foundations	Day 1 – 3
MODULE 1 Introduction to C Language	Day 4 – 10
MODULE 2 Conditional Statements and Loops	Day 11 – 19
MODULE 3 Arrays and Strings	Day 20 – 28
MODULE 4 Functions and Modular Programming	Day 29 – 35
MODULE 5 Structures and Unions	Day 36 – 41
MODULE 6 Pointers and File Processing	Day 42 – 48
CAPSTONE Mini-Project + Exam Revision	Day 49 – 50

Syllabus Coverage Map

This material follows your official syllabus exactly. The table below shows how the daily classes map to each syllabus module and Course Outcome (CO). Module 0 is an added beginner primer (history + setup) so a first-semester student can start from zero.

Syllabus Module	Topics (from syllabus)	Covered in	CO
1. Introduction to C Language	Character set, variables, identifiers, data types, operators, expressions, constants, I/O, simple programs	Days 4–10	CO1
2. Conditional Statements & Loops	Decision making, relational/logical operators, if / if-else / switch, while / do-while / for, nested & infinite loops, structured programming	Days 11–19	CO2
3. Arrays and Strings	1-D & 2-D arrays, searching, insertion/deletion, largest/smallest, matrix add/multiply/transpose, strings, sparse matrix	Days 20–28	CO2
4. Functions & Modular Programming	Top-down design, functions, library functions, prototypes, parameters, call by value/reference, recursion, arrays as arguments	Days 29–35	CO3
5. Structures and Unions	Structure variables, initialization, assignment, nested structures, structures & functions, arrays of structures, unions	Days 36–41	CO4
6. Pointers & File Processing	Address operators, pointer declaration/arithmetic, pointers with functions & arrays, pointer arrays, files, modes, read/write/close	Days 42–48	CO4, CO5
Integrated Application	Mini-project combining all concepts + full exam revision	Days 49–50	CO6

MODULE 0

Getting Started — Foundations

Before we write any C code, we build the ground floor: what a computer program actually is, where the C language came from, how a C program is turned into something the computer can run, and how to install and use the tools. Nothing here is assumed — we start from zero.

DAY 1 What is a Program? History of C & Why We Learn It

Goal of the day: Understand what a computer program is, learn the story of the C language, and know why C is still the best first language to learn.

1. What is a computer and a program?

A **computer** is a machine that is very fast and very obedient, but not clever on its own. It can only do exactly what it is told, step by step. The list of steps we give it is called a **program**. Writing that list of steps in a language the computer can understand is called **programming** or **coding**.

IN SIMPLE WORDS

A program is just a **recipe**. A cooking recipe tells a person: take rice, add water, boil for 10 minutes. A computer program tells the computer: take two numbers, add them, show the result on screen. The computer follows your recipe blindly — so if the recipe is wrong, the result is wrong.

2. Programming languages: high-level vs low-level

The computer's own language is **machine language** — only 0s and 1s (called binary). Humans find that almost impossible to write. So we use **high-level languages** like C, which look much closer to English and mathematics. A special program later translates our C code into the 0s and 1s the machine understands.

Type	What it looks like	Who uses it
Machine language	10110000 01100001	The CPU directly
Assembly language	MOV AL, 61h	Very low-level system work
High-level (C, Python, Java)	c = a + b;	Almost all programmers

3. The history of C (a short story)

C was created in the early **1970s** (around 1972) by **Dennis Ritchie** at **Bell Laboratories** in the USA. It grew out of two earlier languages called **BCPL** and **B** — that is literally why the new, improved language was simply named **C** (the letter after B).

- C was built mainly to rewrite the **UNIX operating system**. Before C, operating systems were written in assembly, which was slow to write and tied to one machine. C let them write UNIX once and run it on many machines.

- In **1978**, Ritchie and Brian Kernighan wrote the famous book *The C Programming Language* (often called “K&R”). This became the first standard.
- In **1989/1990** the language was officially standardised by ANSI and ISO (this version is called **ANSI C** or C89/C90). Later standards followed: **C99**, **C11**, **C17**, and **C23**.

Why this matters to you: almost every modern language you will ever hear of — C++, Java, C#, JavaScript, Python, PHP — borrowed its basic look and rules from C. Learn C well, and half of every other language already feels familiar.

4. Why learn C first?

- **It teaches the fundamentals.** C makes you understand how memory, variables and logic really work, instead of hiding it.
- **It is fast and small.** C programs run very close to the hardware, so they are extremely efficient.
- **It is everywhere.** Operating systems (Windows, Linux, macOS cores), databases, embedded chips in cars, ACs, and washing machines are largely written in C.
- **It is the parent language.** Once you know C, learning C++, Java or Python becomes much easier.

Tip: You do not need to be good at higher mathematics to learn C. You mainly need **logical thinking** and **practice**. If you can plan the steps to make a cup of tea, you can learn to program.

Day 1 — Quick Revision

- A **program** is a step-by-step set of instructions for the computer.
- C was created by **Dennis Ritchie** at **Bell Labs** around **1972** to build the UNIX operating system.
- C is a **high-level** language that is close to English but still very fast.
- We learn C first because it teaches the real foundations of programming.

DAY 2 How a C Program Runs + Installing the Tools

Goal of the day: Understand how source code becomes a running program, and install a compiler and editor so you can start coding.

1. Source code, compiler and executable

The C code **you write** is called the **source code**. It is saved in a file ending with **.c** (for example, `hello.c`). But the computer cannot run **.c** files directly — remember, it only understands 0s and 1s. So we need a translator program called a **compiler**.

IN SIMPLE WORDS

Think of the **compiler** as a translator at a meeting. You speak in C (a human-friendly language). The compiler translates your whole speech into machine language (0s and 1s) and hands the computer a finished, ready-to-run file called an **executable** (on Windows this ends in **.exe**).

2. The four steps of building a C program

When you 'compile' a C program, four stages happen behind the scenes. You do not run them by hand — one command does it all — but it helps to know what is going on.

Stage	What happens
1. Preprocessing	Lines starting with <code>#</code> (like <code>#include</code>) are handled. It pastes in needed code and expands macros.
2. Compilation	Your C code is translated into assembly , then into object code (machine code), file ending <code>.o</code> .
3. Linking	Your object code is joined with library code (for example the code behind <code>printf</code>) to make one file.
4. Loading & Running	The final executable is loaded into memory and the CPU runs it.

Compiler vs Interpreter: C uses a **compiler** — it translates the *whole* program once, in advance, into a fast executable. Languages like Python use an **interpreter** — it translates and runs the program *line by line* each time. Compiling makes C programs faster to run.

3. What you need to install

You need two things: a **compiler** (the translator) and an **editor / IDE** (a comfortable place to type code). An **IDE** (Integrated Development Environment) bundles the editor, compiler button, and error messages together. Here are the common beginner-friendly choices.

Option	What it gives you	Best for
Code::Blocks (with MinGW)	Editor + GCC compiler in one download	Windows beginners — easiest all-in-one
Dev-C++	Small, simple IDE with a compiler	Very low-end Windows PCs
VS Code + GCC/MinGW	A powerful modern editor; compiler added separately	Students who want a professional setup
Online compiler	Nothing to install; runs in a browser	Trying code instantly / weak laptop

Easiest path (recommended for this course): Code::Blocks

1. Go to the Code::Blocks website and download the version whose name includes **mingw-setup** (this includes the GCC compiler, so you do not install it separately).
2. Run the installer and keep clicking **Next** with the default options.
3. Open Code::Blocks. Go to **File** → **New** → **Empty File**, and you are ready to type your first program (Day 3).

If you use an online compiler (no installation)

- Open a site such as an online C compiler (for example, programiz, onlinegdb, or replit).
- Type your code in the box, press **Run**, and see the output instantly. Great for practising anywhere.

For VS Code (professional setup, optional)

1. Install **VS Code** (the editor).
2. Install **MinGW-w64** (this gives you the `gcc` compiler) and add it to your system PATH.
3. Install the **C/C++ extension** inside VS Code.
4. Verify the compiler by opening a terminal and typing `gcc --version`.

4. Compiling from the command line (good to know)

Even with an IDE, it is worth seeing the raw commands. If you have GCC installed, you compile and run like this:

Tip: `-o hello` means “output the result into a file called hello”. If you skip `-o`, GCC names the executable `a.out` by default.

Day 2 — Quick Revision

- You write **source code** (.c). A **compiler** translates it into an **executable** the computer can run.
- The build has four stages: **preprocess** → **compile** → **link** → **run**.
- C is **compiled** (translated fully in advance), unlike Python which is **interpreted** (line by line).
- The simplest setup for this course is **Code::Blocks with MinGW**, which includes the GCC compiler.

DAY 3 Your First C Program — Structure & Hello World

Goal of the day: Write, understand, compile and run your very first C program, and learn what every line means.

1. The classic first program

It is a tradition that your first program in any language just prints the words **Hello, World!** on the screen. Here it is in C:

2. Understanding every single line

Line	What it means
<code>#include <stdio.h></code>	A preprocessor instruction. It brings in the Standard Input/Output library so we can use <code>printf</code> . <code>stdio.h</code> = <i>standard input output header</i> .
<code>int main()</code>	Every C program starts running from main . It is the entry point . <code>int</code> means main will return an integer number when it finishes.
<code>{ ... }</code>	The curly braces mark the beginning and end of main's body — the block of statements to run.
<code>printf("Hello, World!");</code>	<code>printf</code> is a ready-made function that prints text to the screen. The text goes inside double quotes.
<code>return 0;</code>	Ends main and reports 0 to the operating system, meaning “the program finished successfully.”

Watch out: Every statement inside `main` must end with a **semicolon** `;`. Forgetting the semicolon is the number-one beginner error. The semicolon is like a full stop at the end of a sentence.

3. The general structure of a C program

Almost every C program you write this semester follows this shape:

IN SIMPLE WORDS

Read it top to bottom like this: “Bring in the tools I need (`#include`). Then start at `main` . Do the steps between the braces, one after another. Finally, `return 0` to say I finished cleanly.”

4. Comments — notes for humans

A **comment** is text the compiler **ignores**. It is there to explain your code to yourself and others. C has two styles:

Tip: Use comments to explain *why* you did something, not to repeat the obvious. Good comments make your code easy to understand weeks later when you have forgotten the details.

5. Making the program interactive

Printing fixed text is nice, but real programs work with data. Here is a slightly bigger first program that adds two numbers. Do not worry about every detail yet — we cover it all in Module 1. Just see the shape.

The `%d` inside the quotes is a **placeholder** — `printf` replaces it with the value of `sum` . You will master placeholders (called **format specifiers**) in Day 7.

Practice for Day 3

Problem 1: Print your name and course

Write a program that prints your name on one line and your course on the next line.

```
#include <stdio.h>

int main()
{
    printf("Name: Rahul Sharma\n");
    printf("Course: B.Tech CSE, 1st Semester\n");
    return 0;
}
```

OUTPUT

```
Name: Rahul Sharma
Course: B.Tech CSE, 1st Semester
```

How it works: The `\n` is a special **newline** code. It moves the cursor to the next line, just like pressing Enter. Two `printf` statements each print one line.

Problem 2: Print a small box using symbols

Print a simple rectangle made of star symbols.

```
#include <stdio.h>

int main()
{
    printf("*****\n");
    printf(" *      *\n");
    printf("*****\n");
    return 0;
}
```

OUTPUT

```
*****
 *      *
*****
```

How it works: This shows that `printf` prints text *exactly* as written between the quotes. Spaces and stars are printed literally; each `\n` ends the current line.

Day 3 — Quick Revision

- Every C program needs `#include <stdio.h>` and an `int main()` function.
- `printf("...")` prints text; every statement ends with a **semicolon**.
- `\n` moves to a new line; `return 0;` ends the program successfully.
- **Comments** (`//` or `/* */`) are notes for humans and are ignored by the compiler.

MODULE 1

Introduction to C Language

Now we learn the building blocks of the C language itself: the character set, keywords, identifiers, variables and data types, constants, input/output, and operators. By the end of this module you can write real programs that take input, do calculations, and print results. (Maps to CO1.)

DAY 4 Character Set, Tokens, Keywords & Identifiers

Goal of the day: Learn the alphabet of C — the characters and word-types (tokens) that every C program is built from.

1. The C character set

Just as English is built from letters, digits and punctuation, C programs are built from a fixed **character set**:

- **Letters:** A–Z and a–z (C is **case-sensitive**: `Total` and `total` are different).
- **Digits:** 0–9.
- **Special symbols:** `+ - * / % = < > ( ) { } [ ] ; , . # & | ! ? :` and more.
- **White space:** spaces, tabs and new lines (used to separate words and make code readable).

2. Tokens — the smallest units

When the compiler reads your program, it breaks it into small meaningful pieces called **tokens**. There are exactly six kinds of tokens in C.

Token type	Meaning	Example
Keywords	Reserved words with a fixed meaning	<code>int, if, while, return</code>
Identifiers	Names you create for variables/functions	<code>age, total, main</code>
Constants	Fixed values that never change	<code>10, 3.14, 'A'</code>
Strings	Text inside double quotes	<code>"Hello"</code>
Operators	Symbols that do operations	<code>+, -, *, ==</code>
Special symbols	Punctuation with a role	<code>;, { } ()</code>

IN SIMPLE WORDS

A token is like a **word** in a sentence. The compiler first splits your program into words (tokens), then checks whether they are arranged in a way that makes sense (grammar).

3. Keywords

Keywords are reserved words that already have a special meaning in C. You are **not allowed** to use them as your own names. Standard C (C89) has 32 keywords. You will meet most of them this semester:

4. Identifiers — the names you invent

An **identifier** is a name you give to a variable, function, etc. You have freedom here, but you must follow **rules**:

Rule	Correct	Wrong
Must start with a letter or underscore <code>_</code>	<code>total</code> , <code>_count</code>	<code>2marks</code>
Then may contain letters, digits, underscores	<code>marks1</code> , <code>roll_no</code>	<code>roll-no</code> (dash not allowed)
No spaces allowed	<code>first_name</code>	<code>first name</code>
Cannot be a keyword	<code>number</code>	<code>int</code> (reserved)
Case-sensitive	<code>Age</code> $\neq$ <code>age</code>	—

Tip: Choose **meaningful** names. `marks` and `average` are far better than `m` and `a`. Good names make a program read almost like English.

Practice for Day 4

Problem 1: Valid or invalid identifier?

Decide which of these are valid C identifiers: `total_marks`, `9pin`, `float`, `_temp`, `my age`, `Roll1`.

```
total_marks  -> VALID   (letters + underscore)
9pin         -> INVALID (cannot start with a digit)
float        -> INVALID (float is a keyword)
_temp        -> VALID   (may start with an underscore)
my age       -> INVALID (spaces are not allowed)
Roll1        -> VALID   (letter, then letters/digits)
```

How it works: Go through the rules one by one: start character, allowed characters, no spaces, and not a keyword. This kind of question is common in exams, so learn the four rules by heart.

Day 4 — Quick Revision

- The C **character set** = letters, digits, special symbols and white space.
- A program is a stream of **tokens**: keywords, identifiers, constants, strings, operators, special symbols.
- **Keywords** (32 of them) are reserved; **identifiers** are names you create following the naming rules.
- C is **case-sensitive**.

DAY 5 Variables & Built-in Data Types

Goal of the day: Learn how to store data in named boxes (variables) and choose the right type of box for each kind of value.

1. What is a variable?

A **variable** is a named box in the computer's memory that holds a value. The value can **change** while the program runs — that is why it is called a *variable*.

IN SIMPLE WORDS

Imagine labelled boxes on a shelf. One box is labelled **age** and holds the number 18. Later you can open that box and put 19 inside. The label (name) stays the same; the value inside can change.

2. Declaring and using variables

Before you use a variable you must **declare** it — tell C its name and what type of data it will hold.

3. The basic built-in data types

C gives you a small set of **built-in data types**. You pick the type that fits the data.

Type	Stores	Example value	Format specifier	Typical size
<code>int</code>	Whole numbers	<code>25, -7, 0</code>	<code>%d</code>	4 bytes
<code>float</code>	Decimal numbers	<code>3.14, -0.5</code>	<code>%f</code>	4 bytes
<code>double</code>	Bigger/more precise decimals	<code>3.1415926</code>	<code>%lf</code>	8 bytes
<code>char</code>	A single character	<code>'A', '9', '#'</code>	<code>%c</code>	1 byte

A `char` actually stores a small number (its **ASCII code**). For example the character `'A'` is stored as the number **65**. That is why characters can be used in arithmetic, as you will see later.

4. Type modifiers: signed, unsigned, short, long

You can put words in front of a type to change its range:

- **unsigned** — only 0 and positive values (doubles the positive range). e.g. `unsigned int`.
- **short / long** — use less or more memory. e.g. `long int` for very large whole numbers.

5. The sizeof operator

You can ask C how many bytes a type uses with `sizeof`:

Practice for Day 5

Problem 1: Store and display student details

Store a student's roll number, a marks percentage, and a section (single letter), then print them.

```
#include <stdio.h>
int main()
{
    int roll = 12;
    float percent = 87.5;
    char section = 'B';

    printf("Roll No : %d\n", roll);
    printf("Percent : %f\n", percent);
    printf("Section : %c\n", section);
    return 0;
}
```

OUTPUT

```
Roll No : 12
Percent : 87.500000
Section : B
```

How it works: We picked the correct type for each value: `int` for the whole roll number, `float` for the decimal percentage, and `char` for a single-letter section. Each is printed with its matching format specifier.

Problem 2: Swap the day of two variables (using a third box)

Two variables hold 10 and 20. Exchange their values so they become 20 and 10.

```
#include <stdio.h>
int main()
{
    int a = 10, b = 20, temp;
    temp = a;    // save a
    a = b;       // copy b into a
    b = temp;    // copy saved a into b
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

OUTPUT

```
a = 20, b = 10
```

How it works: To swap two boxes you need a temporary third box, exactly like swapping the contents of two glasses needs an empty third glass. We save `a` in `temp`, overwrite `a` with `b`, then put the saved value into `b`.

Day 5 — Quick Revision

- A **variable** is a named memory box; **declare** it before use.
- Core types: `int` (whole), `float` / `double` (decimal), `char` (one character).
- Modifiers `unsigned`, `short`, `long` change the range/size.
- `sizeof` tells you the size of a type in bytes.

DAY 6 Constants and Literals

Goal of the day: Learn how to represent fixed values that must never change, and how to name them.

1. What is a constant / literal?

A **literal** is a fixed value written directly in code, like `10` or `'A'` or `3.14`. A **constant** is a value that is **not allowed to change** once set. Some values in real life never change — the number of days in a week (7), the value of Pi — and constants let us protect them.

2. Kinds of literals

Literal type	Examples	Notes
Integer literal	<code>10</code> , <code>-45</code> , <code>0</code>	Whole numbers. <code>0x1F</code> = hexadecimal, <code>017</code> = octal.
Floating literal	<code>3.14</code> , <code>-0.5</code> , <code>2.0e3</code>	Decimals. <code>2.0e3</code> means $2.0 \times 10^3 = 2000$.
Character literal	<code>'A'</code> , <code>'9'</code> , <code>'\n'</code>	Exactly one character in single quotes.
String literal	<code>"Hello"</code>	Text in double quotes (many characters).

Watch out: Note the quote difference: `'A'` (single quotes) is a **character**. `"A"` (double quotes) is a **string** — they are not the same thing in C.

3. Two ways to make a named constant

Way 1: the `const` keyword

If you later try `PI = 5;`, the compiler gives an **error** — which is good, it protects your value.

Way 2: the `#define` preprocessor directive

IN SIMPLE WORDS

`#define PI 3.14159` tells the preprocessor: “before compiling, find every `PI` in my code and replace it with 3.14159.” It is like a find-and-replace done automatically.

4. Escape sequences (special characters)

Some characters are hard to type directly, so C uses a backslash `\` to write them. These are called **escape sequences**.

Escape	Meaning
<code>\n</code>	New line (move to next line)
<code>\t</code>	Horizontal tab (a gap)
<code>\\</code>	A literal backslash
<code>\"</code>	A double quote inside a string
<code>\'</code>	A single quote
<code>\0</code>	The null character (marks end of a string)

Practice for Day 6

Problem 1: Circle area with a named constant

Use a constant PI to find the area of a circle whose radius is 5.

```
#include <stdio.h>
#define PI 3.14159
int main()
{
    float radius = 5.0, area;
    area = PI * radius * radius;
    printf("Area of circle = %.2f\n", area);
    return 0;
}
```

OUTPUT

Area of circle = 78.54

How it works: `#define` gives PI a name so the formula reads clearly. `%.2f` means "show the float with 2 digits after the decimal point," which rounds 78.539749 to 78.54.

Problem 2: Print a file path and a quote

Print the text: He said "Hi" and the path C:\Users using escape sequences.

```
#include <stdio.h>
int main()
{
    printf("He said \"Hi\"\n");
    printf("Path: C:\\Users\n");
    return 0;
}
```

OUTPUT

```
He said "Hi"
Path: C:\Users
```

How it works: To print a double quote inside a string we write `\"`, and to print a single backslash we write `\\`. The backslash 'escapes' the normal meaning of the next character.

Day 6 — Quick Revision

- A **literal** is a fixed value; a **constant** is a value that cannot change.
- Make constants with `const` or `#define`.
- `'A'` is a character; `"A"` is a string.
- **Escape sequences** like `\n` and `\t` write special characters.

DAY 7 Input & Output — printf and scanf

Goal of the day: Learn how to show results on screen (printf) and take input from the user (scanf) using format specifiers.

1. Output with printf

`printf` ("print formatted") prints text and values. Inside the quotes you put a **format string**, with **format specifiers** as placeholders that get replaced by the values you list after the comma.

2. Common format specifiers

Specifier	Used for	Example
<code>%d</code>	int (decimal integer)	<code>printf("%d", 25);</code>
<code>%f</code>	float / double	<code>printf("%f", 3.14);</code>
<code>%.2f</code>	float with 2 decimals	gives <code>3.14</code>
<code>%c</code>	single character	<code>printf("%c", 'A');</code>
<code>%s</code>	string	<code>printf("%s", "Hi");</code>
<code>%lf</code>	double (in scanf)	reading a double
<code>%u</code>	unsigned int	non-negative integer

3. Input with scanf

`scanf` ("scan formatted") reads values typed by the user and stores them in variables. There is one crucial rule: you must put an **ampersand** `&` before the variable name (except for strings).

Watch out: Forgetting the `&` in `scanf("%d", &age)` is a very common bug. Without `&`, `scanf` does not know *where* to store the value, and your program may crash or behave strangely. You will fully understand why in Module 6 (Pointers).

IN SIMPLE WORDS

The `&` means "the address of" (the location of the box in memory). `printf` shows a value, so it just needs the value. `scanf` must *put* a value into a box, so it needs the box's address — hence `&`.

4. Reading several values at once

Practice for Day 7

Problem 1: Simple calculator (add two numbers)

Ask the user for two integers and print their sum.

```
#include <stdio.h>
int main()
{
    int x, y, sum;
    printf("Enter first number: ");
    scanf("%d", &x);
    printf("Enter second number: ");
    scanf("%d", &y);
    sum = x + y;
    printf("Sum = %d\n", sum);
    return 0;
}
```

OUTPUT

```
Enter first number: 12
Enter second number: 8
Sum = 20
```

How it works: We declare three boxes, read two of them with `scanf` (each with `&`), add them, and print. This is the pattern for almost every input-process-output program.

Problem 2: Read a float and a character

Read a temperature (decimal) and a unit letter, then print them back.

```
#include <stdio.h>
int main()
{
    float temp;
    char unit;
    printf("Enter temperature: ");
    scanf("%f", &temp);
    printf("Enter unit (C/F): ");
    scanf(" %c", &unit); // note the space before %c
    printf("Temperature = %.1f %c\n", temp, unit);
    return 0;
}
```

OUTPUT

```
Enter temperature: 36.6
Enter unit (C/F): C
Temperature = 36.6 C
```

How it works: Use `%f` to read a float and `%c` for a character. The **space before `%c`** (" `%c` ") is a small but important trick: it tells `scanf` to skip any leftover Enter key from the previous input.

Day 7 — Quick Revision

- `printf` shows output; `scanf` reads input.
- **Format specifiers:** `%d` int, `%f` float, `%c` char, `%s` string.
- In `scanf`, put `&` before each numeric/char variable.
- Add a space before `%c` in `scanf` to skip the leftover newline.

DAY 8 Arithmetic Operators & Expressions

Goal of the day: Do calculations in C and understand how expressions are evaluated, including the tricky integer division.

1. The arithmetic operators

Operator	Meaning	Example (a=7, b=2)	Result
+	Addition	<code>a + b</code>	9
-	Subtraction	<code>a - b</code>	5
*	Multiplication	<code>a * b</code>	14
/	Division	<code>a / b</code>	3 (see below!)
%	Modulus (remainder)	<code>a % b</code>	1

2. The integer division trap

Watch out: When you divide two **integers**, C throws away the fractional part. So `7 / 2` is **3**, not 3.5! This surprises almost every beginner.

IN SIMPLE WORDS

If you want the decimal answer, make at least one number a **float** (write `7.0` instead of `7`). The `%` (modulus) operator gives the **remainder** and only works with integers.

3. Uses of the modulus operator

- **Even or odd?** If `n % 2 == 0`, `n` is even.
- **Last digit of a number:** `n % 10`.
- **Is `x` divisible by `y`?** If `x % y == 0`.

4. Operator precedence (order of operations)

Like in mathematics (BODMAS), C does `*`, `/`, `%` **before** `+` and `-`. Use parentheses to force a different order.

Practice for Day 8

Problem 1: Sum, product and average of two numbers

Read two integers and print their sum, product, and average (as a decimal).

```
#include <stdio.h>
int main()
{
    int a, b;
    scanf("%d %d", &a, &b);
    printf("Sum      = %d\n", a + b);
    printf("Product = %d\n", a * b);
    printf("Average = %.2f\n", (a + b) / 2.0);
    return 0;
}
```

OUTPUT

```
Input: 10 5
Sum      = 15
Product = 50
Average = 7.50
```

How it works: For the average we divide by `2.0` (a float), not `2`, so C keeps the decimal part. If we wrote `(a+b)/2`, the answer would be an integer (7).

Problem 2: Find the last digit and check even/odd

Read a number, print its last digit and whether it is even or odd (just compute, we print with a trick for now).

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    printf("Last digit = %d\n", n % 10);
    printf("n %% 2 = %d (0 means even, 1 means odd)\n", n % 2);
    return 0;
}
```

OUTPUT

```
Input: 347
Last digit = 7
n % 2 = 1 (0 means even, 1 means odd)
```

How it works: `n % 10` gives the remainder after dividing by 10, which is always the last digit. `n % 2` is 0 for even numbers and 1 for odd. (To print a literal `%` sign we write `%%`.)

Day 8 — Quick Revision

- Arithmetic operators: `+` `-` `*` `/` `%`.
- **Integer / integer throws away decimals:** use a float to keep them.
- `%` gives the remainder — great for even/odd and last-digit tricks.
- `*` `/` `%` run before `+` `-`; parentheses override the order.

DAY 9 More Operators: Relational, Logical, Assignment, Increment

Goal of the day: Learn the operators used for comparisons and decisions, plus handy shortcuts for updating variables.

1. Relational operators (comparisons)

These compare two values and give a result of **1 (true)** or **0 (false)**. They are the heart of decision making (Module 2).

Operator	Meaning	Example (a=5,b=8)	Result
<code>==</code>	equal to	<code>a == b</code>	0 (false)
<code>!=</code>	not equal to	<code>a != b</code>	1 (true)
<code>></code>	greater than	<code>a > b</code>	0
<code><</code>	less than	<code>a < b</code>	1
<code>>=</code>	greater or equal	<code>a >= 5</code>	1
<code><=</code>	less or equal	<code>b <= 8</code>	1

Watch out: `=` and `==` are different! `=` **assigns** a value (puts it into a box). `==` **compares** two values. Mixing them up is a classic bug.

2. Logical operators (combine conditions)

Operator	Name	True when	Example
<code>&&</code>	AND	both sides are true	<code>age>=18 && citizen==1</code>
<code> </code>	OR	at least one side is true	<code>day==6 day==7</code>
<code>!</code>	NOT	flips true/false	<code>!(x==0)</code>

IN SIMPLE WORDS

Think of AND / OR like real life. "You can enter if you have a ticket **AND** you are over 12." Both must be true. "You get a discount if you are a student **OR** a senior." Any one is enough.

3. Assignment shortcuts

These update a variable in a shorter way:

Shortcut	Same as	Meaning
<code>x += 5</code>	<code>x = x + 5</code>	add 5 to x
<code>x -= 3</code>	<code>x = x - 3</code>	subtract 3
<code>x *= 2</code>	<code>x = x * 2</code>	double x
<code>x /= 4</code>	<code>x = x / 4</code>	divide by 4
<code>x %= 3</code>	<code>x = x % 3</code>	remainder by 3

4. Increment (++) and decrement (--)

`++` adds 1; `--` subtracts 1. There are two forms:

Pre vs Post: `++a` (pre) changes `a` first, then uses it. `a++` (post) uses the old value first, then changes `a`. This difference matters only when the increment is part of a bigger expression.

5. Type conversion

When different types mix, C converts the smaller to the bigger (implicit conversion). You can also convert on purpose with a **cast** in parentheses:

Practice for Day 9

Problem 1: Compare two numbers with relational operators

Read two numbers and print the results of a few comparisons (as 1/0).

```
#include <stdio.h>
int main()
{
    int a, b;
    scanf("%d %d", &a, &b);
    printf("a == b : %d\n", a == b);
    printf("a > b : %d\n", a > b);
    printf("a != b : %d\n", a != b);
    return 0;
}
```

OUTPUT

```
Input: 5 8
a == b : 0
a > b : 0
a != b : 1
```

How it works: Each comparison gives 1 (true) or 0 (false). For 5 and 8: they are not equal (so `==` is 0, `!=` is 1) and 5 is not greater than 8 (so `>` is 0).

Problem 2: Correct average using a cast

Three subject marks are 70, 80 and 85. Find the average correctly (with decimals).

```
#include <stdio.h>
int main()
{
    int m1 = 70, m2 = 80, m3 = 85;
    int total = m1 + m2 + m3; // 235
    float average = (float) total / 3;
    printf("Average = %.2f\n", average);
    return 0;
}
```

OUTPUT

```
Average = 78.33
```

How it works: Total is 235. Writing `total / 3` alone would give 78 (integer division). Casting with `(float)` keeps the decimals, giving 78.33.

Day 9 — Quick Revision

- **Relational** operators (`==` `!=` `>` `<` `>=` `<=`) give 1 (true) or 0 (false).
- **Logical** operators: `&&` (AND), `||` (OR), `!` (NOT).
- Shortcuts: `+=` `-=` `*=` `/=` ; and `++` `/` `--` to add/subtract 1.
- Use a **cast** like `(float)x` to convert a type on purpose.

DAY 10 Module 1 — Practice & Recap

Goal of the day: Combine everything from Module 1 (variables, I/O, operators) into complete, useful programs.

Mixed practice problems

Problem 1: Area and perimeter of a rectangle

Read length and breadth; print area and perimeter.

```
#include <stdio.h>
int main()
{
    float length, breadth;
    scanf("%f %f", &length, &breadth);
    printf("Area      = %.2f\n", length * breadth);
    printf("Perimeter = %.2f\n", 2 * (length + breadth));
    return 0;
}
```

OUTPUT

```
Input: 5 3
Area      = 15.00
Perimeter = 16.00
```

How it works: Straight application of formulas: $\text{area} = l \times b$, $\text{perimeter} = 2(l+b)$. We used floats so decimal inputs also work.

Problem 2: Convert Celsius to Fahrenheit

Read a temperature in Celsius and convert to Fahrenheit using $F = C \times 9/5 + 32$.

```
#include <stdio.h>
int main()
{
    float c, f;
    scanf("%f", &c);
    f = c * 9 / 5 + 32;
    printf("%.1f C = %.1f F\n", c, f);
    return 0;
}
```

OUTPUT

Input: 37
37.0 C = 98.6 F

How it works: Because `c` is a float, `c * 9 / 5` keeps decimals correctly. If `c` were an int, `9/5` would still be safe here because it is multiplied first, but using a float removes all doubt.

Problem 3: Total seconds into hours, minutes, seconds

Read a total number of seconds and break it into hours, minutes and seconds.

```
#include <stdio.h>
int main()
{
    int total, h, m, s;
    scanf("%d", &total);
    h = total / 3600;           // whole hours
    m = (total % 3600) / 60;    // leftover seconds -> minutes
    s = total % 60;            // final leftover seconds
    printf("%d h %d m %d s\n", h, m, s);
    return 0;
}
```

OUTPUT

Input: 3725
1 h 2 m 5 s

How it works: This is a perfect use of `/` and `%`. Dividing by 3600 gives whole hours. The remainder (`% 3600`) is the leftover seconds, which we turn into minutes by dividing by 60, and so on.

Problem 4: Simple interest

Read principal, rate and time; compute simple interest = $(P \times R \times T) / 100$.

```
#include <stdio.h>
int main()
{
    float p, r, t, si;
    scanf("%f %f %f", &p, &r, &t);
    si = (p * r * t) / 100;
    printf("Simple Interest = %.2f\n", si);
    return 0;
}
```

OUTPUT

Input: 1000 5 2
Simple Interest = 100.00

How it works: A direct formula program: read three values in one `scanf`, plug into the formula, print with two decimals.

Problem 5: Swap two numbers WITHOUT a third variable

Exchange two numbers using only arithmetic (no temp box).

```
#include <stdio.h>
int main()
{
    int a = 4, b = 9;
    a = a + b;    // a = 13
    b = a - b;    // b = 13 - 9 = 4 (old a)
    a = a - b;    // a = 13 - 4 = 9 (old b)
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

OUTPUT

a = 9, b = 4

How it works: A clever trick: by adding then subtracting, the two values trade places without any extra box. It shows how much you can do with just arithmetic operators.

Module 1 — Key Takeaways

- A C program = headers + `main` + statements ending in semicolons.
- Store data in **variables** of the right **type**; protect fixed values with **constants**.
- **printf/scanf** handle output/input; remember `&` in `scanf`.
- Master **operators** — especially integer division, `%`, and `==` vs `=`.

Tip: Exam pointer: questions on integer division results, `%` output, identifier validity, and format specifiers are extremely common. Make sure you can predict the output of small expressions by hand.

MODULE 2

Conditional Statements and Loops

Programs become powerful when they can make decisions and repeat work. This module covers if / if-else / switch for choices, and while / do-while / for loops for repetition, plus nested loops and structured programming. (Maps to CO2.)

DAY 11 Decision Making & the if Statement

Goal of the day: Make your program choose different actions based on a condition, using the if statement.

1. Why decisions?

Real programs must react differently to different situations: pass or fail, adult or minor, discount or no discount. For this we use **conditions** — questions that are either **true** or **false** — built from the relational and logical operators you learned in Day 9.

2. The if statement

IN SIMPLE WORDS

Read `if` as plain English: “IF this is true, then do the things inside the braces. Otherwise, skip them.”

If the input were 30, the condition `marks >= 40` is false, so “You passed!” is skipped, and only “Program finished.” prints.

3. Combining conditions

Watch out: A very common mistake: writing `if (marks = 40)` with a single `=`. That **assigns** 40 to marks and is always treated as true. To compare, you must use `==`.

Tip: If there is only **one** statement inside the if, the braces are optional — but using braces always is a good, safe habit that prevents bugs.

Practice for Day 11

Problem 1: Check if a number is positive

Read a number; print a message if it is positive.

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    if (n > 0)
        printf("%d is positive\n", n);
    return 0;
}
```

OUTPUT

```
Input: 7
7 is positive
```

How it works: The single condition `n > 0` controls whether the message prints. For 0 or a negative number, nothing prints.

Problem 2: Eligible to vote?

Read an age; if 18 or above, print that the person can vote.

```
#include <stdio.h>
int main()
{
    int age;
    scanf("%d", &age);
    if (age >= 18)
        printf("You are eligible to vote.\n");
    return 0;
}
```

OUTPUT

```
Input: 21
You are eligible to vote.
```

How it works: A direct translation of the rule “age 18 or more” into `age >= 18`.

Day 11 — Quick Revision

- `if (condition) { ... }` runs its block only when the condition is **true**.
- Conditions use relational (`>` `<` `==` `!=`) and logical (`&&` `||` `!`) operators.
- Use `==` to compare (not `=`).

DAY 12 if-else, else-if Ladder & Nested if

Goal of the day: Handle two-way and multi-way decisions cleanly.

1. if-else — two paths

IN SIMPLE WORDS

if-else means “if the condition is true do THIS, otherwise do THAT.” Exactly one of the two runs.

2. The else-if ladder — many paths

When there are several possibilities (like grades), chain conditions with **else if**. C checks them top to bottom and runs the **first** one that is true, then stops.

Order matters! C stops at the first true condition. If you put **marks >= 40** at the top, everyone above 40 would get grade D. Always arrange from the strictest condition to the loosest.

3. Nested if — an if inside an if

Practice for Day 12

Problem 1: Largest of three numbers

Read three numbers and print the largest.

```
#include <stdio.h>
int main()
{
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);
    if (a >= b && a >= c)
        printf("Largest = %d\n", a);
    else if (b >= a && b >= c)
        printf("Largest = %d\n", b);
    else
        printf("Largest = %d\n", c);
    return 0;
}
```

OUTPUT

Input: 12 45 27
Largest = 45

How it works: Each branch checks whether one number is at least as big as the other two using **&&**. If a is not the largest and b is not, then c must be, so the final **else** handles it.

Problem 2: Even or odd

Read a number and say whether it is even or odd.

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    if (n % 2 == 0)
        printf("%d is even\n", n);
    else
        printf("%d is odd\n", n);
    return 0;
}
```

OUTPUT

```
Input: 13
13 is odd
```

How it works: A number is even when it leaves no remainder on division by 2, i.e. `n % 2 == 0`. Otherwise it is odd.

Problem 3: Positive, negative or zero

Classify a number as positive, negative, or zero.

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    if (n > 0)    printf("Positive\n");
    else if (n < 0) printf("Negative\n");
    else        printf("Zero\n");
    return 0;
}
```

OUTPUT

```
Input: 0
Zero
```

How it works: Three mutually exclusive cases handled by an else-if ladder. Exactly one branch runs.

Day 12 — Quick Revision

- `if-else` chooses between two paths.
- The **else-if ladder** handles many cases; C runs the first true one and stops.
- Arrange conditions from strictest to loosest.
- You can **nest** an if inside another if.

DAY 13 The switch Statement

Goal of the day: Use switch for clean multi-way choices based on a single value.

1. When to use switch

When you compare **one variable** against several fixed values (like a menu choice 1/2/3), `switch` is cleaner than a long else-if ladder.

2. The parts of a switch

- **case X:** — if the variable equals X, start running from here.
- **break;** — jumps out of the switch. **Very important!**
- **default:** — runs if no case matched (like the final `else`).

Watch out: If you **forget** `break`, execution “falls through” into the next case and keeps running until it hits a break or the end. Sometimes this is used on purpose, but usually a missing break is a bug.

IN SIMPLE WORDS

Think of `switch` as a signboard with numbered doors. The value picks a door (`case`). `break` is the exit; without it you walk straight into the next room.

3. switch limitations

- It only works with **integer** or **char** values (not float, not strings).
- Cases must be **constants** (you cannot write `case x > 5:`).

Practice for Day 13

Problem 1: Simple calculator with switch

Read two numbers and an operator (+, -, *, /) and perform the operation.

```
#include <stdio.h>
int main()
{
    float a, b;
    char op;
    scanf("%f %c %f", &a, &op, &b);
    switch (op)
    {
        case '+': printf("%.2f\n", a + b); break;
        case '-': printf("%.2f\n", a - b); break;
        case '*': printf("%.2f\n", a * b); break;
        case '/': printf("%.2f\n", a / b); break;
        default : printf("Unknown operator\n");
    }
    return 0;
}
```

OUTPUT

```
Input: 6 * 4
24.00
```

How it works: The `char op` selects the case. Because each case ends in `break`, only the matching operation runs. Note switching on a `char` is allowed.

Problem 2: Days of the week

Read a number 1-7 and print the day name.

```
#include <stdio.h>
int main()
{
    int d;
    scanf("%d", &d);
    switch (d)
    {
        case 1: printf("Monday\n");    break;
        case 2: printf("Tuesday\n");   break;
        case 3: printf("Wednesday\n"); break;
        case 4: printf("Thursday\n");  break;
        case 5: printf("Friday\n");     break;
        case 6: printf("Saturday\n");   break;
        case 7: printf("Sunday\n");     break;
        default: printf("Invalid day\n");
    }
    return 0;
}
```

OUTPUT

Input: 5
Friday

How it works: A textbook menu-style switch. Any number outside 1–7 falls to `default`.

Day 13 — Quick Revision

- `switch` compares one integer/char value against several **constant** cases.
- Each case usually ends with `break`; to avoid fall-through.
- `default`: handles the no-match situation.

DAY 14 The while Loop

Goal of the day: Repeat a block of statements as long as a condition stays true.

1. Why loops?

Suppose you must print “Hello” 100 times. Writing 100 `printf` lines is silly. A **loop** repeats a block automatically. The `while` loop repeats **as long as** its condition is true.

IN SIMPLE WORDS

Every loop needs three things: a **start** (`i = 1`), a **condition** to keep going (`i <= 5`), and an **update** (`i++`). If you forget the update, the condition never becomes false and the loop runs forever.

Watch out: An **infinite loop** happens when the condition never becomes false. If you write `while(i<=5)` but forget `i++`, the program hangs. Press Ctrl+C to stop a stuck program.

2. Reading the loop as a story

The three steps run like this: check condition → if true, run the body → do the update → check condition again → ... until the condition is false, then the loop ends.

Practice for Day 14

Problem 1: Print numbers 1 to N

Read N and print all numbers from 1 to N.

```
#include <stdio.h>
int main()
{
    int n, i = 1;
    scanf("%d", &n);
    while (i <= n)
    {
        printf("%d ", i);
        i++;
    }
    return 0;
}
```

OUTPUT

Input: 6
1 2 3 4 5 6

How it works: Start at 1, keep going while `i <= n`, and add 1 each time. When i becomes n+1 the condition fails and the loop stops.

Problem 2: Sum of first N natural numbers

Read N and find $1 + 2 + \dots + N$.

```
#include <stdio.h>
int main()
{
    int n, i = 1, sum = 0;
    scanf("%d", &n);
    while (i <= n)
    {
        sum += i;    // add current i to the running total
        i++;
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

OUTPUT

Input: 5
Sum = 15

How it works: We keep a **running total** in **sum**, starting at 0. Each turn we add the current **i**. For N=5: $1+2+3+4+5 = 15$.

Problem 3: Reverse the digits of a number

Read a number and print its digits in reverse.

```
#include <stdio.h>
int main()
{
    int n, digit;
    scanf("%d", &n);
    while (n > 0)
    {
        digit = n % 10;    // grab last digit
        printf("%d", digit);
        n = n / 10;        // remove last digit
    }
    printf("\n");
    return 0;
}
```

OUTPUT

Input: 1234
4321

How it works: $n \% 10$ pulls off the last digit; $n / 10$ chops it away. Repeating this prints the digits from last to first, which is the reverse.

Day 14 — Quick Revision

- `while` repeats a block **as long as** the condition is true.
- Every loop needs a **start**, a **condition**, and an **update**.
- Forgetting the update creates an **infinite loop**.

DAY 15 The do-while Loop

Goal of the day: Use do-while when the body must run at least once before checking the condition.

1. do-while: check at the bottom

A `while` loop checks the condition **before** running the body — so the body might run zero times. A `do-while` runs the body **first**, then checks — so it always runs **at least once**.

Watch out: A `do-while` ends with a **semicolon** after the condition: `} while (i<=5);`. Forgetting it is a common error.

2. while vs do-while

Feature	while	do-while
Condition checked	Before body	After body
Minimum runs	0 times	At least 1 time
Best for	May not need to run at all	Menus, input that must be asked once

IN SIMPLE WORDS

Use `do-while` when you must do something at least once — like showing a menu, or asking for a password — before you can even check the condition.

Practice for Day 15

Problem 1: Repeat a menu until user quits

Show a menu; keep looping until the user enters 0.

```
#include <stdio.h>
int main()
{
    int choice;
    do
    {
        printf("1.Play  2.Settings  0.Exit : ");
        scanf("%d", &choice);
        if (choice == 1) printf("Playing...\n");
        else if (choice == 2) printf("Opening settings...\n");
    } while (choice != 0);
    printf("Goodbye!\n");
    return 0;
}
```

OUTPUT

```
1.Play  2.Settings  0.Exit : 1
Playing...
1.Play  2.Settings  0.Exit : 0
Goodbye!
```

How it works: The menu must appear at least once, so **do-while** is perfect. The loop repeats until the user types 0, which makes **choice != 0** false.

Problem 2: Sum digits of a number

Read a number and add up all its digits.

```
#include <stdio.h>
int main()
{
    int n, sum = 0;
    scanf("%d", &n);
    do
    {
        sum += n % 10;    // add last digit
        n /= 10;         // remove it
    } while (n > 0);
    printf("Digit sum = %d\n", sum);
    return 0;
}
```

OUTPUT

```
Input: 512
Digit sum = 8
```

How it works: We add the last digit and chop it off each turn. For 512: $2 + 1 + 5 = 8$. Using do-while means it also works correctly for a single-digit number entered once.

Day 15 — Quick Revision

- **do-while** runs the body first, then checks — so it runs **at least once**.
- End it with a **semicolon** after the condition.
- Great for menus and “ask at least once” situations.

DAY 16 The for Loop

Goal of the day: Master the for loop — the most common loop for counting a known number of times.

1. The for loop packs everything into one line

A **for** loop puts the **start**, **condition** and **update** together on one line, which makes counting loops very tidy.

IN SIMPLE WORDS

Read it as: “Start i at 1. Keep looping while $i \leq 5$. After each loop, do $i++$.” It is the same three parts as a while loop, just gathered in one neat line.

2. Order of execution

1. The **start** runs once, at the very beginning.
2. The **condition** is checked. If false, the loop ends.
3. If true, the **body** runs.

4. The **update** runs. Then go back to step 2.

3. Counting backwards and in steps

Practice for Day 16

Problem 1: Multiplication table

Print the multiplication table of a number N (1 to 10).

```
#include <stdio.h>
int main()
{
    int n, i;
    scanf("%d", &n);
    for (i = 1; i <= 10; i++)
        printf("%d x %d = %d\n", n, i, n * i);
    return 0;
}
```

OUTPUT

```
Input: 3
3 x 1 = 3
3 x 2 = 6
...
3 x 10 = 30
```

How it works: The loop variable `i` goes 1 to 10, and each turn we print `n × i`. A classic first for-loop program.

Problem 2: Factorial of a number

Compute $N! = 1 \times 2 \times \dots \times N$.

```
#include <stdio.h>
int main()
{
    int n, i;
    long fact = 1;           // start product at 1, use long for big values
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        fact *= i;           // multiply running product by i
    printf("%d! = %ld\n", n, fact);
    return 0;
}
```

OUTPUT

Input: 5
5! = 120

How it works: A **running product** that starts at 1 (not 0, or everything becomes 0). We multiply by each i : $1 \times 1 \times 2 \times 3 \times 4 \times 5 = 120$.

Problem 3: Count multiples of 3 up to N

Count how many numbers from 1 to N are divisible by 3.

```
#include <stdio.h>
int main()
{
    int n, i, count = 0;
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        if (i % 3 == 0)
            count++;
    printf("Count = %d\n", count);
    return 0;
}
```

OUTPUT

Input: 10
Count = 3

How it works: We loop through every number and use `i % 3 == 0` to test divisibility, bumping a counter each time. For 1–10 the multiples of 3 are 3, 6, 9 — so count is 3.

Day 16 — Quick Revision

- `for (start; condition; update)` gathers all loop controls in one line.
- Best when you know **how many times** to repeat.

- You can count up, down, or in steps.

DAY 17 Nested Loops & Pattern Printing

Goal of the day: Put one loop inside another to work with rows and columns, and print shapes.

1. What is a nested loop?

A **nested loop** is a loop inside another loop. The **outer** loop usually controls **rows**, and the **inner** loop controls **columns**. For every single turn of the outer loop, the inner loop runs completely.

IN SIMPLE WORDS

Think of a cinema hall. The outer loop is the **row number**; the inner loop counts the **seats** in that row. You finish all seats in row 1, print a newline, then move to row 2, and so on.

2. A triangle pattern

The key idea: the inner loop runs **i** times in row **i**. So row 1 has 1 star, row 2 has 2 stars, and so on.

Practice for Day 17

Problem 1: Number triangle

Print a triangle where each row shows numbers 1..row.

```
#include <stdio.h>
int main()
{
    int i, j;
    for (i = 1; i <= 5; i++)
    {
        for (j = 1; j <= i; j++)
            printf("%d ", j);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

How it works: Same as the star triangle, but we print the counter **j** instead of a star. In row **i**, **j** runs from 1 to **i**.

Problem 2: Multiplication grid

Print a 3x3 grid of products (row × column).

```
#include <stdio.h>
int main()
{
    int i, j;
    for (i = 1; i <= 3; i++)
    {
        for (j = 1; j <= 3; j++)
            printf("%d\t", i * j);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

1	2	3
2	4	6
3	6	9

How it works: For each cell we print `i * j`. The `\t` (tab) neatly aligns the columns. This shows how nested loops naturally map to a 2-D table.

Problem 3: Right-aligned star pyramid (with spaces)

Print a centered pyramid of stars for 4 rows.

```
#include <stdio.h>
int main()
{
    int i, j, rows = 4;
    for (i = 1; i <= rows; i++)
    {
        for (j = 1; j <= rows - i; j++) // leading spaces
            printf(" ");
        for (j = 1; j <= 2 * i - 1; j++) // stars
            printf("*");
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
  *
 ***
*****
*****
```

How it works: Two inner loops: the first prints spaces so the row is pushed right, the second prints an odd number of stars (1, 3, 5, 7). Combining spaces and stars gives the pyramid shape.

Day 17 — Quick Revision

- A **nested loop** is a loop inside a loop — outer = rows, inner = columns.
- For each outer turn, the inner loop runs fully.
- Patterns come from choosing how many times the inner loop runs per row.

DAY 18 break, continue, Infinite Loops & Structured Programming

Goal of the day: Control loops precisely with break and continue, and learn the discipline of structured programming.

1. break — leave the loop immediately

As soon as `i` reaches 5, `break` stops the whole loop, so 5 to 10 are never printed.

2. continue — skip to the next turn

IN SIMPLE WORDS

`break` = leave the loop for good. `continue` = skip the leftover work in this turn and jump to the next turn. Here, even numbers are skipped, so only odds print.

3. Infinite loops (on purpose and by accident)

Watch out: Sometimes an infinite loop is intentional (a server waiting for requests). But an **accidental** infinite loop — usually from forgetting to update the loop variable — will freeze your program. Always make sure the loop moves toward its end.

4. Structured programming

Structured programming is a clean way of writing programs using only three kinds of building blocks, avoiding messy jumps (the old `goto`). Every program can be built from just these:

Structure	Meaning	In C
Sequence	Run steps one after another	normal statements
Selection	Choose a path	<code>if</code> , <code>if-else</code> , <code>switch</code>
Iteration	Repeat	<code>while</code> , <code>do-while</code> , <code>for</code>

Benefits of structured programming: code is **easier to read, test, debug and maintain**. This course teaches you to write structured code from day one — avoid `goto`.

Practice for Day 18

Problem 1: Stop at the first number divisible by 7

Print numbers from 1 upward, but stop as soon as you find one divisible by 7.

```
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 100; i++)
    {
        if (i % 7 == 0)
        {
            printf("First multiple of 7 is %d\n", i);
            break;
        }
    }
    return 0;
}
```

OUTPUT

```
First multiple of 7 is 7
```

How it works: The loop keeps going until it meets the condition, then `break` ends the search. This is the standard 'search and stop' pattern.

Problem 2: Sum only the odd numbers from 1 to N

Add up the odd numbers, skipping evens with continue.

```
#include <stdio.h>
int main()
{
    int n, i, sum = 0;
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        if (i % 2 == 0)
            continue;    // skip even numbers
        sum += i;
    }
    printf("Sum of odds = %d\n", sum);
    return 0;
}
```

OUTPUT

Input: 7
Sum of odds = 16

How it works: `continue` jumps past the `sum += i` for even numbers, so only odds (1+3+5+7 = 16) are added.

Day 18 — Quick Revision

- `break` exits a loop entirely; `continue` skips to the next turn.
- `while(1)` is an intentional infinite loop — use `break` to exit.
- **Structured programming** = Sequence + Selection + Iteration; avoid `goto`.

DAY 19 Module 2 — Practice & Recap

Goal of the day: Solve mixed decision-and-loop problems that are common in exams.

Problem 1: Prime number check

Read a number and decide whether it is prime (divisible only by 1 and itself).

```
#include <stdio.h>
int main()
{
    int n, i, isPrime = 1;    // assume prime until proven otherwise
    scanf("%d", &n);
    if (n <= 1)
        isPrime = 0;
    for (i = 2; i <= n / 2; i++)
    {
        if (n % i == 0)      // found a divisor
        {
            isPrime = 0;
            break;
        }
    }
    if (isPrime) printf("%d is prime\n", n);
    else        printf("%d is not prime\n", n);
    return 0;
}
```

OUTPUT

Input: 13
13 is prime

How it works: We test whether any number from 2 up to $n/2$ divides n . If one does, n is not prime and we stop early with `break`. A flag variable `isPrime` remembers the result.

Problem 2: Sum of digits until single digit (digital root)

Keep summing digits until only one digit remains.

```
#include <stdio.h>
int main()
{
    int n, sum;
    scanf("%d", &n);
    while (n >= 10)           // while more than one digit
    {
        sum = 0;
        while (n > 0)
        {
            sum += n % 10;
            n /= 10;
        }
        n = sum;              // repeat with the new sum
    }
    printf("Digital root = %d\n", n);
    return 0;
}
```

OUTPUT

Input: 9875

Digital root = 2

How it works: 9875 -> 9+8+7+5 = 29 -> 2+9 = 11 -> 1+1 = 2. The outer loop repeats the digit-sum until a single digit is left. A neat example of a loop inside a loop working on numbers.

Problem 3: Fibonacci series up to N terms

Print the first N Fibonacci numbers (each is the sum of the previous two).

```
#include <stdio.h>
int main()
{
    int n, i;
    long a = 0, b = 1, next;
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        printf("%ld ", a);
        next = a + b;    // next term
        a = b;           // shift the window forward
        b = next;
    }
    return 0;
}
```

OUTPUT

Input: 8

0 1 1 2 3 5 8 13

How it works: We keep the last two numbers in **a** and **b**. Each turn we print **a**, compute the next term, and slide the pair forward. This is a very common interview and exam question.

Problem 4: Count and reverse using a while loop

Read a number and print how many digits it has.

```
#include <stdio.h>
int main()
{
    int n, count = 0;
    scanf("%d", &n);
    if (n == 0) count = 1;
    while (n > 0)
    {
        count++;
        n /= 10;
    }
    printf("Number of digits = %d\n", count);
    return 0;
}
```

OUTPUT

Input: 40507
Number of digits = 5

How it works: Each time we divide by 10 we remove one digit and count it. When n becomes 0 there are no digits left. We handle 0 as a special case (it has one digit).

Module 2 — Key Takeaways

- **Selection:** `if` / `if-else` / `else-if` / `switch` choose what to do.
- **Iteration:** `while` / `do-while` / `for` repeat work; each needs start, condition, update.
- **Nested loops** handle rows & columns and patterns.
- `break` / `continue` fine-tune loops; write **structured** code.

Tip: Exam pointer: prime check, factorial, Fibonacci, digit-reversal, digit-sum and pattern printing appear again and again. Practise them until you can write them without looking.